

20 questions DAX

Contexte de filtre /20



Avant ton entretien Data Analyst / BI. Durée conseillée : 25 min · 3 paliers · corrigé détaillé.

Sur les dashboards Power BI que j'audite en clientèle, la majorité des totaux faux ne viennent pas d'une faute de syntaxe : ils viennent d'un contexte de filtre mal compris (une relation inactive, un ALL oublié, un SUMX remplacé par un SUM par réflexe). Ce test vérifie si tu sais lire un contexte avant d'écrire une mesure.

Repères : 12/20 = bases du contexte acquises, 17/20 = tu peux défendre un dashboard en entretien.

Candidat _____ Date _____ Note _____ /20

Palier 1 : le contexte, la brique de base.

Sept questions sur ce que CALCULATE modifie, la différence entre contexte de ligne et contexte de filtre, et le choix entre SUM et SUMX.

Question 1. Scénario. Un visuel matriciel affiche le CA par Région avec la mesure $CA = \text{SUM}(\text{Ventes}[\text{Montant}])$. Qu'est-ce qui définit le contexte de filtre appliqué à la cellule "Île-de-France" ?

- a) La ligne et la colonne du visuel filtrent la table Ventes via les relations actives.
- b) Uniquement la formule de la mesure, indépendamment du visuel.
- c) L'ordre des colonnes dans la table Ventes.
- d) Le tri appliqué à la colonne Région dans le visuel.

Question 2. Que fait CALCULATE par rapport au contexte de filtre déjà en place ?

- a) Il ignore toujours le contexte de filtre déjà en place.
- b) Il ajoute, remplace ou retire des filtres selon ses arguments, modifiant le contexte de filtre courant.
- c) Il transforme systématiquement le contexte de filtre en contexte de ligne.
- d) Il ne fonctionne que dans les colonnes calculées.

Question 3. Lecture de code. Colonne calculée $\text{Ventes}[\text{MontantLigne}] = \text{Ventes}[\text{PrixUnitaire}] * \text{Ventes}[\text{Quantite}]$. Pourquoi cette formule calcule correctement chaque ligne sans SUMX ?

- a) Parce que DAX applique un SUMX implicite à toute colonne calculée.
- b) Parce que PrixUnitaire et Quantite sont toutes deux numériques.
- c) Parce qu'une colonne calculée s'évalue dans un contexte de ligne : chaque ligne connaît déjà ses propres valeurs.
- d) Parce que le moteur DAX trie la table avant de calculer la colonne.

Question 4. Scénario. La mesure $CA = \text{SUM}(\text{Ventes}[\text{Montant}])$ est placée dans un visuel matriciel par Produit. Pourquoi SUM suffit-il ici, sans itérateur ?

- a) SUM ignore le contexte de filtre posé par le visuel.
- b) SUM ne fonctionne que sur une seule ligne à la fois.
- c) SUM nécessite toujours SUMX pour s'exécuter correctement.
- d) SUM parcourt les lignes visibles dans le contexte de filtre courant et additionne la colonne physique.

Question 5. Meilleur choix. Il n'existe pas de colonne MontantLigne dans Ventes, seulement PrixUnitaire et Quantite. Pour calculer le CA total, quelle mesure est correcte ?

- a) `SUMX(Ventes, Ventes[PrixUnitaire] * Ventes[Quantite])`
- b) `SUM(Ventes[PrixUnitaire]) * SUM(Ventes[Quantite])`
- c) `Ventes[PrixUnitaire] * Ventes[Quantite]`
- d) `SUM(Ventes[PrixUnitaire] * Ventes[Quantite])`

Question 6. Scénario. `MargePct = [Marge] / [CA]` renvoie une erreur sur les produits sans vente ce mois-ci. Quelle correction règle le problème ?

- a) Remplacer / par `MOD([Marge], [CA])`.
- b) Utiliser `DIVIDE([Marge], [CA])`, qui gère nativement la division par zéro.
- c) Ajouter `IF([CA] < 0, 0, [Marge] / [CA])`.
- d) Le problème ne peut pas être corrigé en DAX, il faut nettoyer les données sources.

Question 7. Scénario. `NbClients = DISTINCTCOUNT(Ventes[ClientID])`. Pourquoi le total région ne correspond-il pas à la somme des NbClients par produit ?

- a) Parce que la table Ventes contient des doublons de clients.
- b) Parce que le total région utilise une relation inactive.
- c) Parce que `DISTINCTCOUNT` est une mesure non additive : un même client acheteur de plusieurs produits n'est compté qu'une fois au niveau région.
- d) Parce que `DISTINCTCOUNT` ne fonctionne pas dans un total général.

Palier 2 : les pièges qui faussent un total en silence.

Sept questions où la mesure semble marcher, mais le chiffre peut être faux.

Question 8. Spot the error. Une colonne calculée MargeLigne et une mesure [Marge] portent la même formule de base. Pourquoi la colonne ne réagit-elle pas aux filtres du visuel comme la mesure ?

- a) Parce que les colonnes calculées ne peuvent pas contenir de multiplication.
- b) Parce qu'une mesure est toujours plus rapide qu'une colonne calculée.
- c) Parce que la colonne calculée est stockée dans une table différente.
- d) Parce que la colonne calculée est figée au rafraîchissement dans un contexte de ligne, alors que la mesure se recalculé dans le contexte de filtre du visuel.

Question 9. Scénario. Une relation Ventes vers Calendrier sur la date de livraison est inactive au profit de la relation sur la date de commande. Comment forcer un calcul sur la date de livraison ?

- a) `CALCULATE([CA], USERELATIONSHIP(Ventes[DateLivraison], Calendrier[Date]))`
- b) Rendre la relation active dans le modèle, ce qui désactivera l'autre relation automatiquement.
- c) Utiliser `RELATED(Calendrier[Date])` directement dans la mesure.
- d) Ce n'est pas possible sans dupliquer la table Calendrier.

Question 10. Scénario. Relation Produits (1) vers Ventes (plusieurs) en filtre à sens unique. Un segment (slicer) posé sur une colonne de Ventes filtre-t-il la table Produits ?

- a) Oui, toute relation filtre automatiquement dans les deux sens par défaut.
- b) Non, en sens unique le filtre ne remonte pas vers Produits; il faudrait activer un filtre bidirectionnel (both).
- c) Oui, mais seulement si la colonne filtrée est numérique.
- d) Non, aucune relation ne peut jamais filtrer une table côté un.

Question 11. Lecture de code. Motif classique pour un pourcentage du total par produit : `CALCULATE([CA], ALL(Produits))` au dénominateur. Que fait `ALL(Produits)` ici ?

- a) Il filtre uniquement le produit sélectionné dans le visuel.
- b) Il duplique le contexte de filtre du produit courant.
- c) Il retire les filtres posés sur la table Produits pour recalculer un total général non filtré par produit.
- d) Il trie les produits par ordre alphabétique avant le calcul.

Question 12. Scénario. Une mesure `SAMEPERIODLASTYEAR([CA], Calendrier[Date])` renvoie des blancs. Calendrier n'est pas marquée comme table de dates. Quelle est la cause la plus probable ?

- a) Il manque une colonne Année dans Calendrier.
- b) `SAMEPERIODLASTYEAR` ne fonctionne qu'avec des dates au format texte.
- c) La mesure `[CA]` n'est pas définie correctement.
- d) Les fonctions time intelligence exigent une table de dates continue, marquée Date Table et reliée par une colonne date propre.

Question 13. Lecture de code. `CALCULATE([CA], FILTER(Ventes, Ventes[Montant] > 500))` dans une mesure. Quelle est la valeur de `FILTER` par rapport à un filtre direct sur une colonne ?

- a) `FILTER` évalue la condition ligne par ligne dans Ventes et retourne une table, utile pour une logique dépassant un simple filtre de colonne.
- b) Il n'y a aucune différence, les deux syntaxes sont strictement équivalentes dans tous les cas.
- c) `FILTER` est toujours plus lent et à éviter dans `CALCULATE`.
- d) `FILTER` ne fonctionne que sur des colonnes de type texte.

Question 14. Scénario. `MargePctMoyenne` doit refléter la marge % moyenne pondérée sur toutes les ventes, pas la moyenne simple des pourcentages ligne à ligne. Quelle formule est correcte ?

- a) `AVERAGE(Ventes[MargePctLigne])`
- b) `DIVIDE(SUMX(Ventes, Ventes[Marge]), SUMX(Ventes, Ventes[Montant]))`
- c) `SUM(Ventes[MargePctLigne])`
- d) `Ventes[Marge] / Ventes[Montant]`

Palier 3 : relations, ALL et time intelligence.

Six questions de raisonnement sous contrainte, là où se joue le 17/20.

Question 15. Scénario. Depuis Ventes (plusieurs), on veut le nom du produit associé à chaque ligne. Depuis Produits (un), on veut la liste des ventes d'un produit. Quelles fonctions utiliser ?

- a) `RELATED` pour la liste de ventes d'un produit, `RELATEDTABLE` pour retrouver un produit depuis une vente.
- b) `RELATED` depuis Ventes vers Produits (plusieurs vers un), `RELATEDTABLE` depuis Produits vers Ventes (un vers plusieurs).
- c) Les deux fonctions sont interchangeable, seul le nom change selon la table de départ.
- d) `RELATED` et `RELATEDTABLE` ne fonctionnent que dans des mesures, jamais dans des colonnes calculées.

Question 16. Scénario. `CALCULATE([CA], KEEPFILTERS(Produits[Categorie] = "Bureautique"))` dans un visuel déjà filtré sur `Categorie = "Informatique"`. Que se passe-t-il ?

- a) Il remplace le filtre existant sur `Categorie` par le nouveau, comme un `CALCULATE` classique.
- b) Il ignore le filtre du visuel et affiche toujours "Bureautique".
- c) Il combine le nouveau filtre avec le filtre existant du visuel au lieu de le remplacer : le résultat est vide car les deux catégories ne peuvent pas être vraies en même temps.
- d) Il supprime tous les filtres de la table Produits.

Question 17. Meilleur choix. Un total par région doit ignorer le filtre pays mais garder le filtre région du visuel. Quelle fonction choisir ?

- a) `ALL(Geo)`, qui retire tous les filtres de Geo y compris celui sur Region.
- b) `ALLSELECTED(Geo)`, réservé aux contextes de segments visuels.
- c) `FILTER(Geo, Geo[Pays] = "France")`.
- d) `ALLEXCEPT(Geo, Geo[Region])`, qui retire tous les filtres de Geo sauf celui posé sur Region.

Question 18. Lecture de code. `CALCULATE([CA], ALL(Ventes))` contre `CALCULATE([CA], ALL(Ventes[Annee]))`.
Quelle différence de portée ?

- a) `ALL(Ventes)` retire tous les filtres de toute la table, `ALL(Ventes[Annee])` ne retire que le filtre posé sur la colonne `Annee`.
- b) Les deux syntaxes ont exactement le même effet, `ALL` agit toujours sur la table entière.
- c) `ALL(Ventes[Annee])` retire plus de filtres que `ALL(Ventes)`, car il cible une colonne précise.
- d) `ALL(Ventes)` ne fonctionne que sur des tables sans relations actives.

Question 19. Scénario. Deux relations bidirectionnelles se croisent entre Ventes, Produits et Promotions. Un total apparaît plus élevé que prévu. Cause la plus probable ?

- a) Le contexte de filtre bidirectionnel ne peut jamais changer un total, seul `CALCULATE` le peut.
- b) Une erreur de saisie dans la table Promotions, sans lien avec les relations.
- c) Un filtre bidirectionnel propage dans une direction inattendue à travers les deux relations et amplifie le contexte de filtre, créant un sur-comptage.
- d) Le moteur DAX additionne automatiquement deux fois toute mesure utilisant `SUMX`.

Question 20. Scénario. La table Calendrier contient des trous (aucune vente certains jours) mais est marquée comme Date Table. La mesure `TOTALYTD([CA], Calendrier[Date])` reste correcte. Pourquoi ?

- a) Parce que `TOTALYTD` ignore les trous de données par défaut, quelle que soit la table de dates.
- b) Parce que `Calendrier` est reliée à `Ventes` par une relation bidirectionnelle.
- c) Parce que `TOTALYTD` recalcule automatiquement les jours manquants dans `Ventes`.
- d) Parce que `Calendrier` est une table de dates continue générée indépendamment des transactions (une ligne par jour civil) : les trous dans `Ventes` n'affectent pas la continuité des dates.

Solutions et explications.

Un point par bonne réponse. Compte ton score, puis relis surtout celles que tu ne peux pas expliquer à voix haute.

1. **a)** Le contexte de filtre d'une cellule de matrice vient toujours du visuel (lignes, colonnes, segments) propagé par les relations actives, pas de la seule formule de la mesure.
2. **b)** CALCULATE modifie le contexte de filtre courant : il peut ajouter, remplacer ou retirer des filtres selon ses arguments, jamais l'ignorer complètement.
3. **c)** Le piège classique est de croire qu'une colonne calculée hérite du contexte de filtre d'un visuel : elle ne connaît que le contexte de ligne de sa propre table.
4. **d)** SUM est une agrégation simple : elle additionne la colonne physique sur les lignes que laisse passer le contexte de filtre courant, sans avoir besoin d'itérer explicitement.
5. **a)** Sans colonne physique pour le produit, il faut un itérateur qui calcule la multiplication ligne par ligne avant de sommer ; multiplier deux SUM agrégés donne un résultat faux.
6. **b)** DIVIDE encapsule la gestion de la division par zéro et renvoie un blanc (ou une valeur par défaut) au lieu de faire planter le visuel comme l'opérateur / brut.
7. **c)** Le piège du DISTINCTCOUNT : il n'est pas additif. Un client acheteur de trois produits compte trois fois au niveau produit, mais une seule fois au niveau région.
8. **d)** Une colonne calculée est figée dans un contexte de ligne au moment du rafraîchissement ; seule une mesure se recalcule dynamiquement dans le contexte de filtre du visuel.
9. **a)** USERELATIONSHIP à l'intérieur de CALCULATE active temporairement une relation inactive pour ce calcul précis, sans toucher au modèle.
10. **b)** Une relation à sens unique ne propage le filtre que dans la direction déclarée ; pour filtrer côté un depuis côté plusieurs, il faut du bidirectionnel (both).
11. **c)** ALL(Produits) retire les filtres posés sur la table Produits, ce qui recalcule le CA total non filtré par produit : le dénominateur du pourcentage.
12. **d)** Les fonctions time intelligence s'appuient sur une table de dates marquée et continue ; sans cette déclaration, le moteur ne sait pas naviguer d'une période à l'autre.
13. **a)** FILTER retourne une table filtrée ligne par ligne, indispensable dès que la condition dépasse un simple test sur une seule colonne (logique combinée, plusieurs colonnes).
14. **b)** Une marge % pondérée est un ratio de sommes, pas une moyenne de ratios : AVERAGE traiterait une petite vente à poids égal avec une grosse vente, ce qui fausse le résultat.
15. **b)** RELATED lit une valeur côté plusieurs vers un (une seule ligne possible), RELATEDTABLE renvoie une table côté un vers plusieurs (plusieurs lignes possibles) : les inverser échoue.
16. **c)** KEEPFILTERS ne remplace pas le filtre existant du visuel, il le combine : avec deux catégories différentes en même temps, le résultat logique est vide.
17. **d)** ALLEXCEPT garde précisément le filtre demandé (Region) tout en retirant les autres filtres de la table ; ALL(Geo) aurait aussi cassé le filtre région du visuel.
18. **a)** ALL sur une table entière retire tous ses filtres ; ALL sur une colonne précise ne retire que le filtre de cette colonne, les autres filtres de la table restent actifs.
19. **c)** Le bidirectionnel propage le contexte de filtre dans les deux sens sur chaque relation ; combiné sur plusieurs tables, il peut créer un chemin de filtrage inattendu et gonfler un total.
20. **d)** Une table de dates marquée est générée indépendamment des transactions (une ligne par jour civil) : les fonctions time intelligence naviguent sur cette continuité, pas sur les dates présentes dans Ventas.

Fait main, en solo.

Si ce test t'a aidé, partage-le autour de toi : il peut débloquent un autre candidat sur son prochain entretien Power BI. Et si tu veux que je traite un sujet DAX précis, dis-le moi en commentaire : c'est là que je pioche mes prochains guides.

Dylan

